YData: Introduction to Data Science

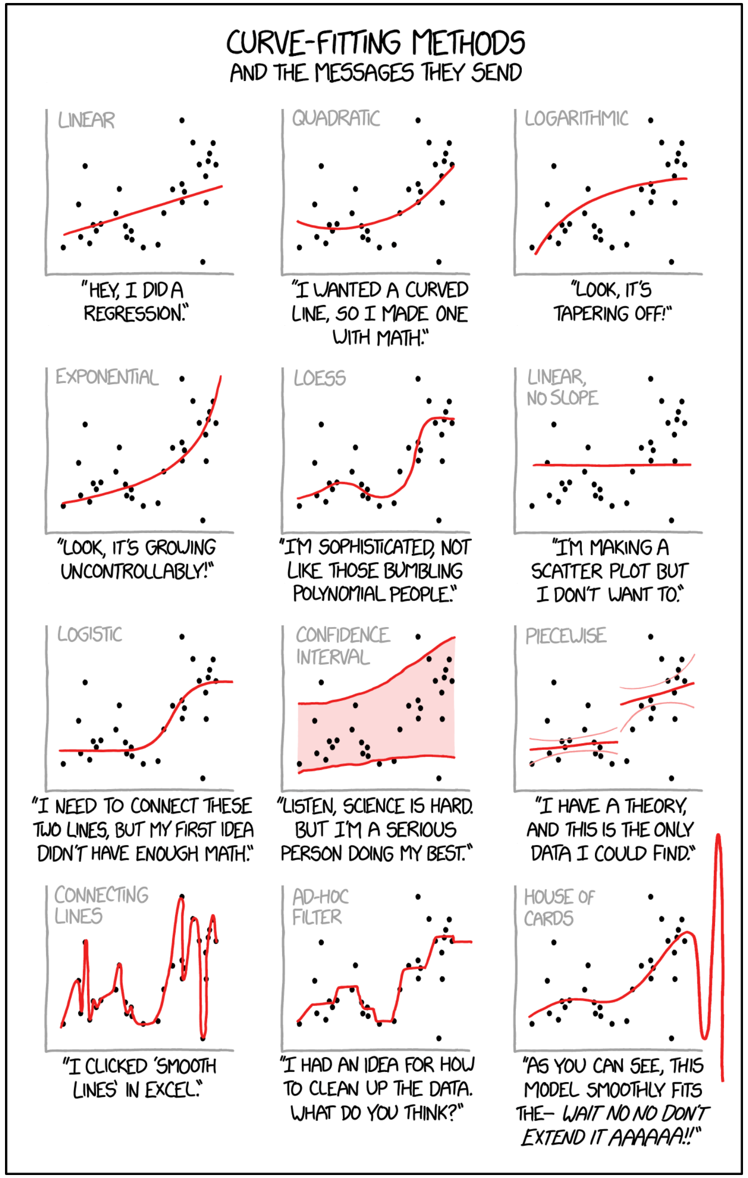
Lecture 23: Linear regression

Overview

Review of classification and the KNN classifier

Feature normalization

Linear regression



Project timeline

~~Friday, April 10th~~

- ~~Projects are due on Gradescope at 11pm~~
- ~~Email a **pdf** of your project to your peer reviewers~~
  - ~~A list of whose paper you will review is posted on Canvas~~
  - ~~Fill out the draft reflection on Canvas~~

Friday, April 17th

- A template for doing your review has been posted
- Jupyter notebook files with your reviews need to be emailed to the authors
- A pdf containing all three reviews needs to be uploaded Gradescope

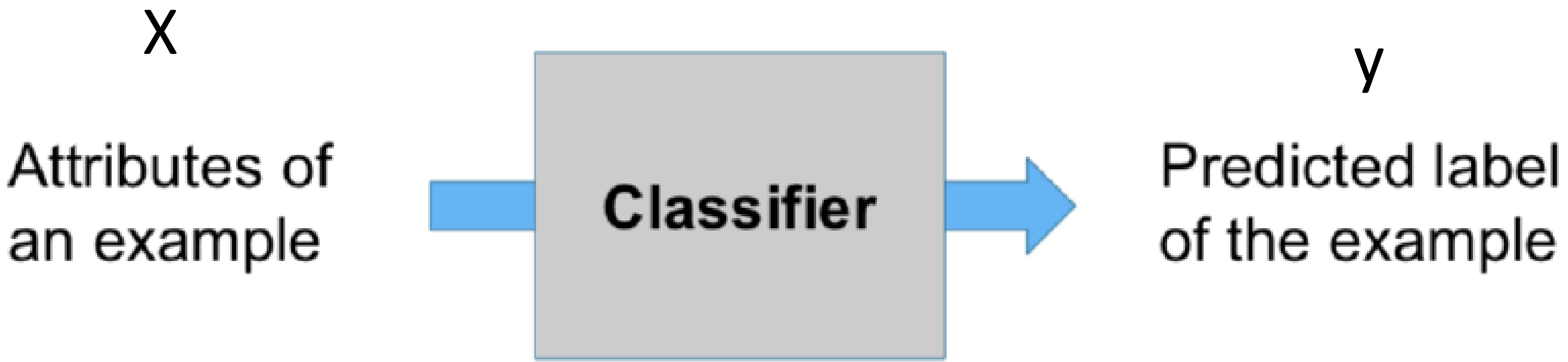
Friday, April 24th

- Project is due on Gradescope
- Add the peer reviews of your project to the Appendix of your project

Homework 9 has been posted

- It is due Sunday April 19th

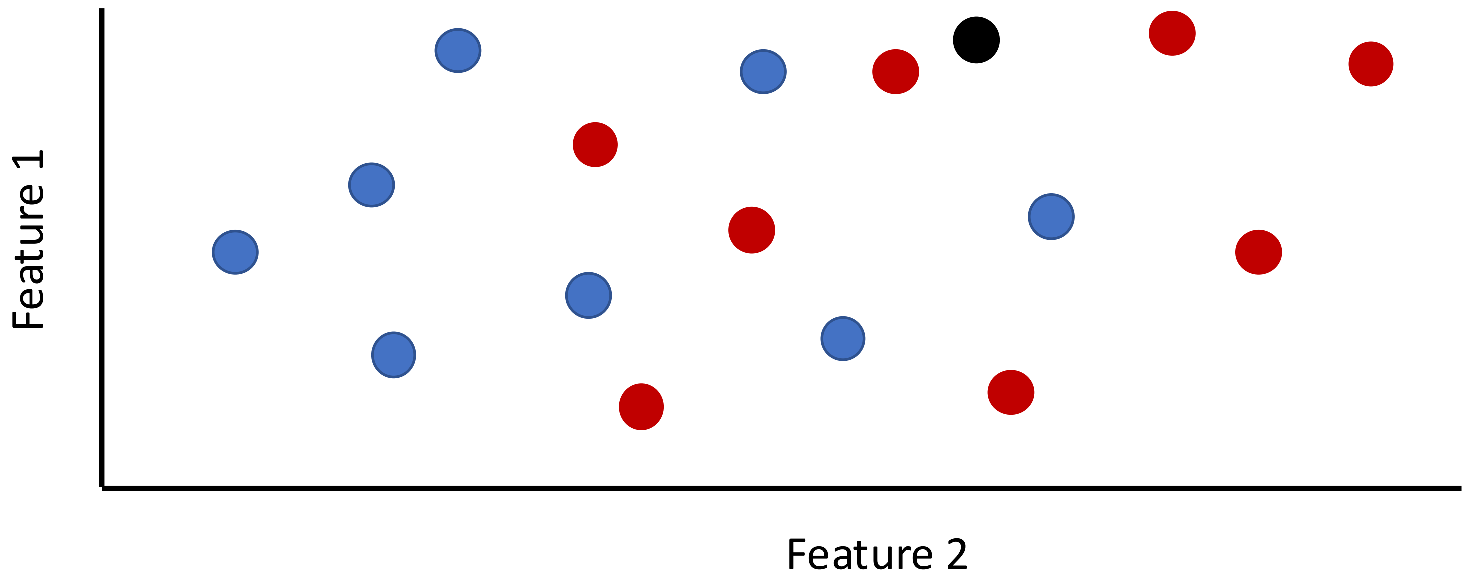
Review: Training a classifier



K-Nearest Neighbor Classifier (KNN)

To classify a point:

- Find its k nearest neighbors
- Take a majority vote of the k nearest neighbors to see which of the two classes appears more often
- Assign the point the class that wins the majority vote



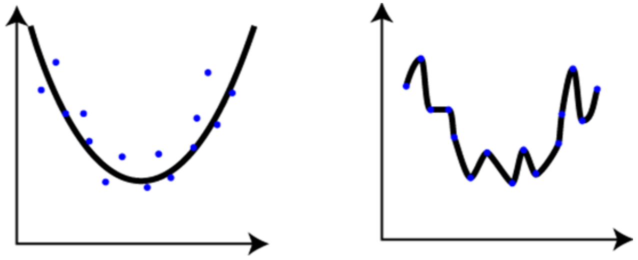
KNN classifiers using scikit-learn

We can fit and evaluate the performance of a KNN classifier using:

```
knn = KNeighborsClassifier(n_neighbors = 1)        # construct a classifier

knn.fit(X_features, y_labels)      # train the classifier

penguin_preditions = knn.predict(X_penguin_features)  # make predictions

np.mean(penguin_preditions == y_penguin_labels)       # get accuracy
```

Review: overfitting

Overfitting occurs when our classifier matches too close to the training data and doesn't capture the true underlying patterns

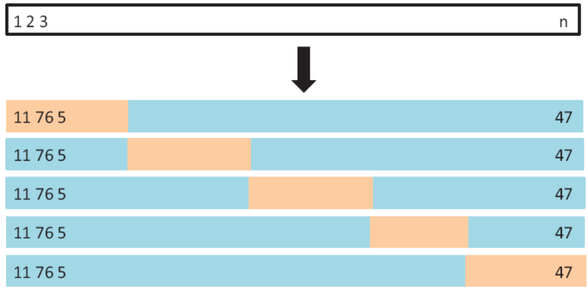


Overfitting song...

To avoid overfitting we split our data into a training and test set

- Classifier learns relationship on training data
- Classifier's performance is evaluated on the test data

We can use k-fold cross-validation to get a better estimate of the test accuracy

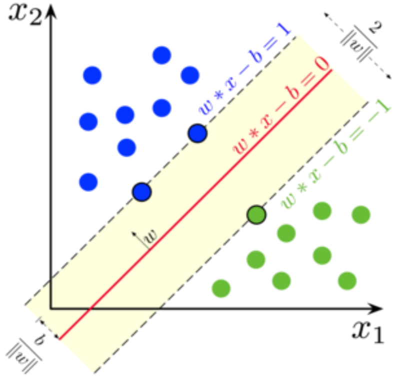


Review: Other classifiers

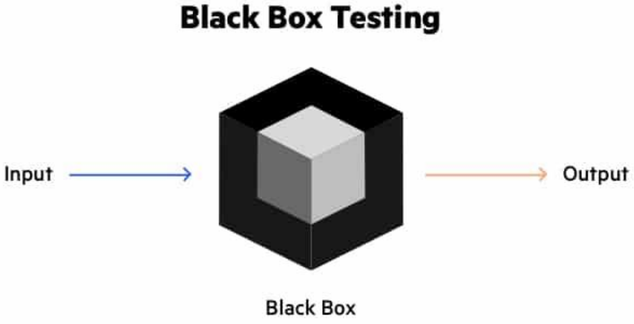
We also built our own KNN classifier

Scikit-learn makes it easy to try out different classifiers get their cross-validation performance

- E.g., SVM, random forests, neural networks, etc.




```
svm = LinearSVC()

scores = cross_val_score(svm,
      X_features,  y_labels, cv = 5)

scores.mean()
```



Review: Steps to build a KNN classifier

We build our KNN classifier by creating a series of functions…

1. euclid_dist(x1, x2)
   - Calculates the Euclidean distance between two points x1 and x2

$$D = \sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2 + (z_0 - z_1)^2}$$

2. get_labels_and_distances(test_point, X_train_features, y_train_labels)
   - Finds the distance between a test point and all the training points

3. classify_point(test_point, k, X_train_features, y_train_labels)
   - Classifies one test point by returning the majority label of the k closest points

4. classify_all_test_data(X_test_data, k, X_train_features, y_train_labels)
   - Classifiers all test points

Let's continue exploring this in Jupyter!

Feature normalization

Review: Distance between two points

Two features x and y: $D = \sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2}$

Three features x, y, and z: $D = \sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2 + (z_0 - z_1)^2}$

- And so on for more features…

It's important the features are standardized

- If not, features that typically have larger values will dominate the distance measurement

Feature normalization

In order to deal with features that are measured on very different scales, we can normalize the features

With a z-score normalization, we normalize each feature to have:

- A mean of 0
- A standard deviation of 1

We can do this in Python using:

```
scalar = StandardScaler()
scalar.fit(X_train)
X_train_transformed = scalar.transform(X_train)
X_test_transformed = scalar.transform(X_test)
```

bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
46.1	15.1	215.0	5100.0
37.3	17.8	191.0	3350.0
51.3	18.2	197.0	3750.0
39.5	16.7	178.0	3250.0
48.7	15.1	222.0	5350.0

$\bar{x}$

s

$$z_i = \frac{x_i - \bar{x}}{s}$$

Feature normalization

To avoid overfitting ("data leakage") we can:

- Calculate the mean and standard deviation on the training
- Apply these means and standard deviations to normalize the training and test sets

To do this in a cross-validation loop, we can use a pipeline:

```
scalar = StandardScaler()
knn = KNeighborsClassifier(n_neighbors = 1)
cv = KFold(n_splits=5)

pipeline = Pipeline([('transformer', scalar), ('estimator', knn)])

scores = cross_val_score(pipeline, X_penguin_features, y_penguin_labels, cv = cv)
```

Let's explore this in Jupyter!

Linear regression

Prediction: regression and classification

We “learn” a function f

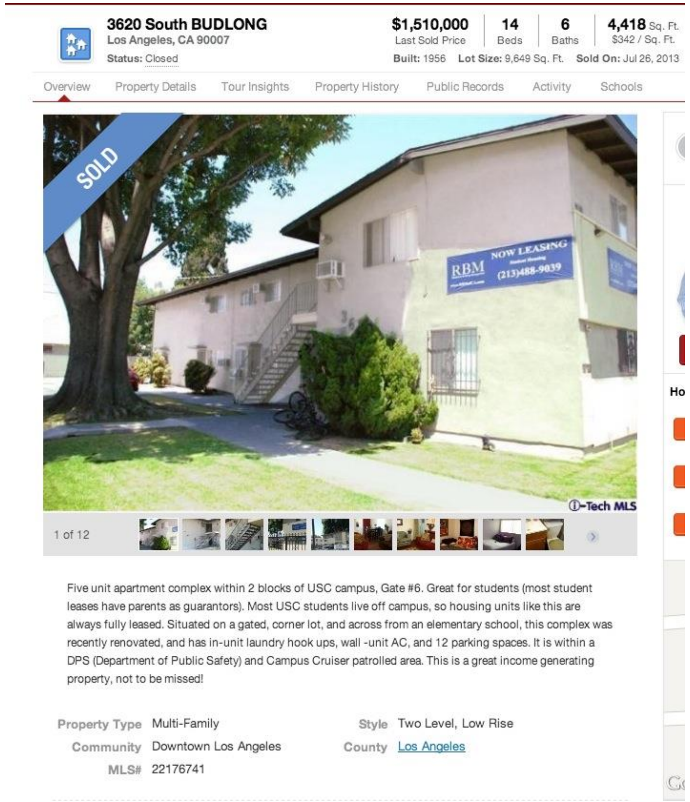
- f(**x**) $\longrightarrow$ y

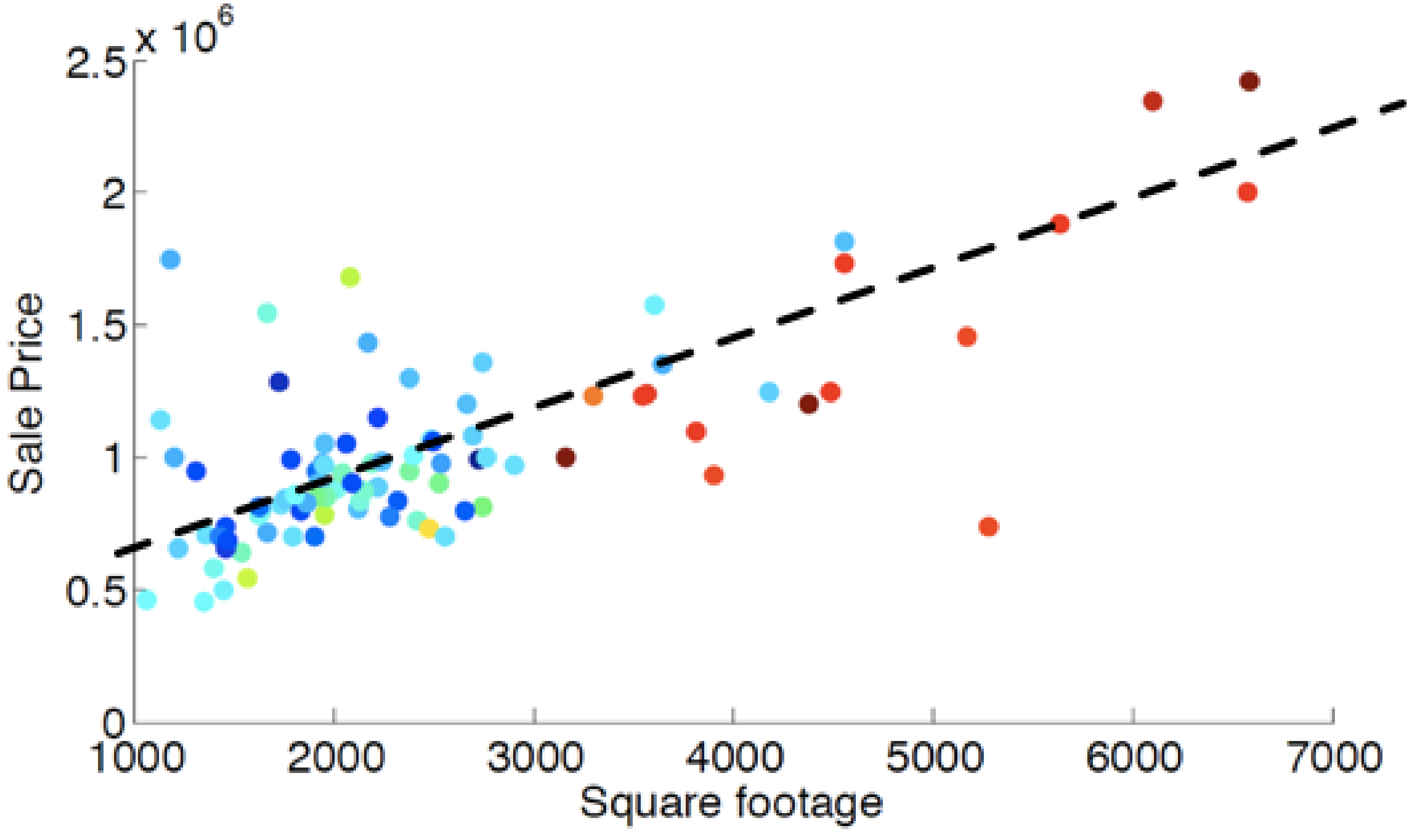
Input: **x** is a data vector of "features"

Output:

- Regression: output is a real number ($y \in R$)
- Classification: output is a categorical variable y_k

Example: predicting house prices





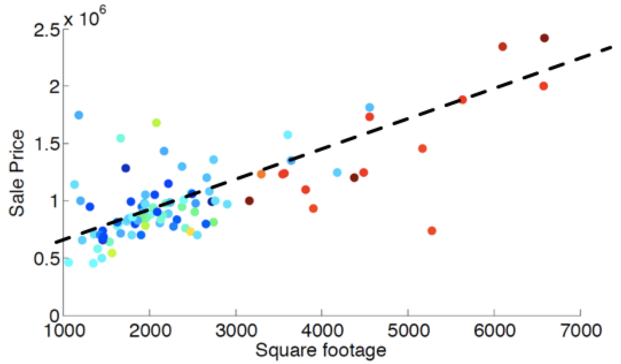
Sale price ≈ price_per_sqft × square_footage + fixed_expense

Regression

Regression is method of using one variable ***x*** ***to predict*** the value of a second variable ***y***

- i.e., $\hat{y} = f(x)$
- Linear regression: $\hat{y} = \text{intercept} + \text{slope} \cdot x$

$$\hat{y} = b_0 + b_1 \cdot x$$

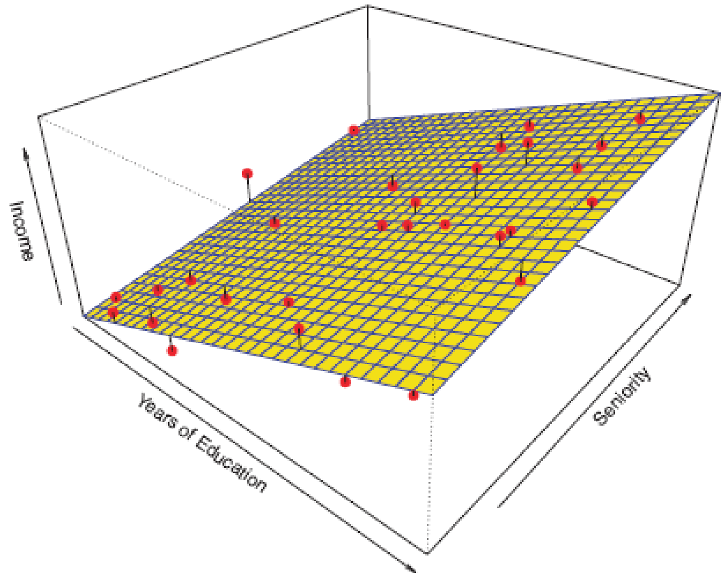


The coefficients for these regression models are found by minimizing root mean square error (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Regression line app

Multiple regression



In multiple regression we try to predict a quantitative response variable *y* using several features $x_1, x_2, \ldots, x_k$

We estimate coefficients using a data set to make predictions ŷ

$$\hat{y} = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \ldots + b_k \cdot x_k$$

Learn the b_i's on the training set. Assess prediction accuracy on test set.

Multiple regression

$$\hat{y} = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \ldots + b_k \cdot x_k$$

There are many uses for multiple regression models including:

- To make predictions as accurately as possible
- To understand which predictors (x) are related to the response variable (y)

Linear regression models in scikit-learn

We can use scikit-learn to create linear regression models

- You can also use the stats models package to do this

```
linear_model = LinearRegression()  # construct a linear regression model

linear_model.fit(X_train_features, y_train)      # train the classifier
```

"Learns" b_0, b_1, ... by minimizing the RMSE on the training data

Numeric values to predict

```
y_preditions = linear_model.predict(X_test_features)   # make predictions

RMSE = np.sqrt(np.mean((y_test - y_predictions)**2))   # get the RMSE
```

Real world example

Rather than predicting stock prices…

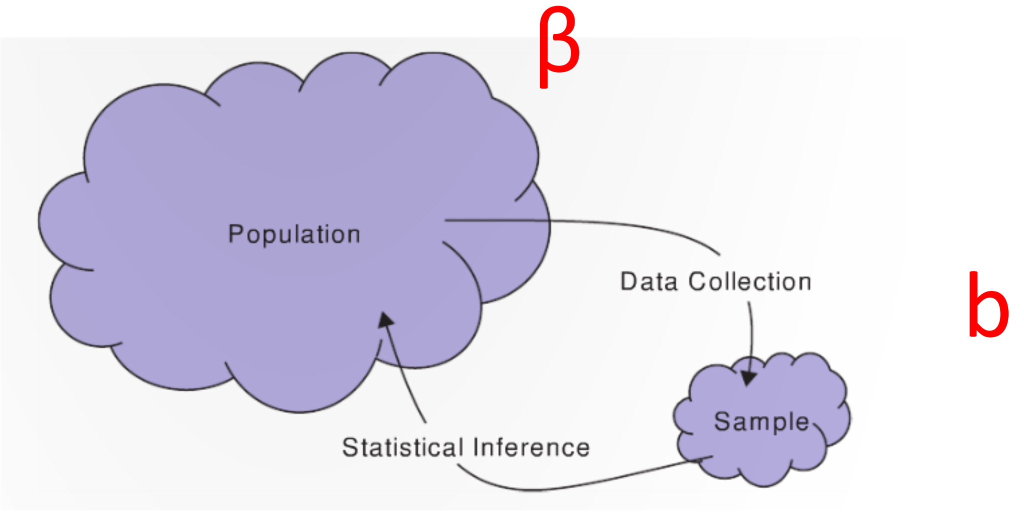
Let’s try this in Jupyter!

Inference for simple linear regression

Inference for simple linear regression

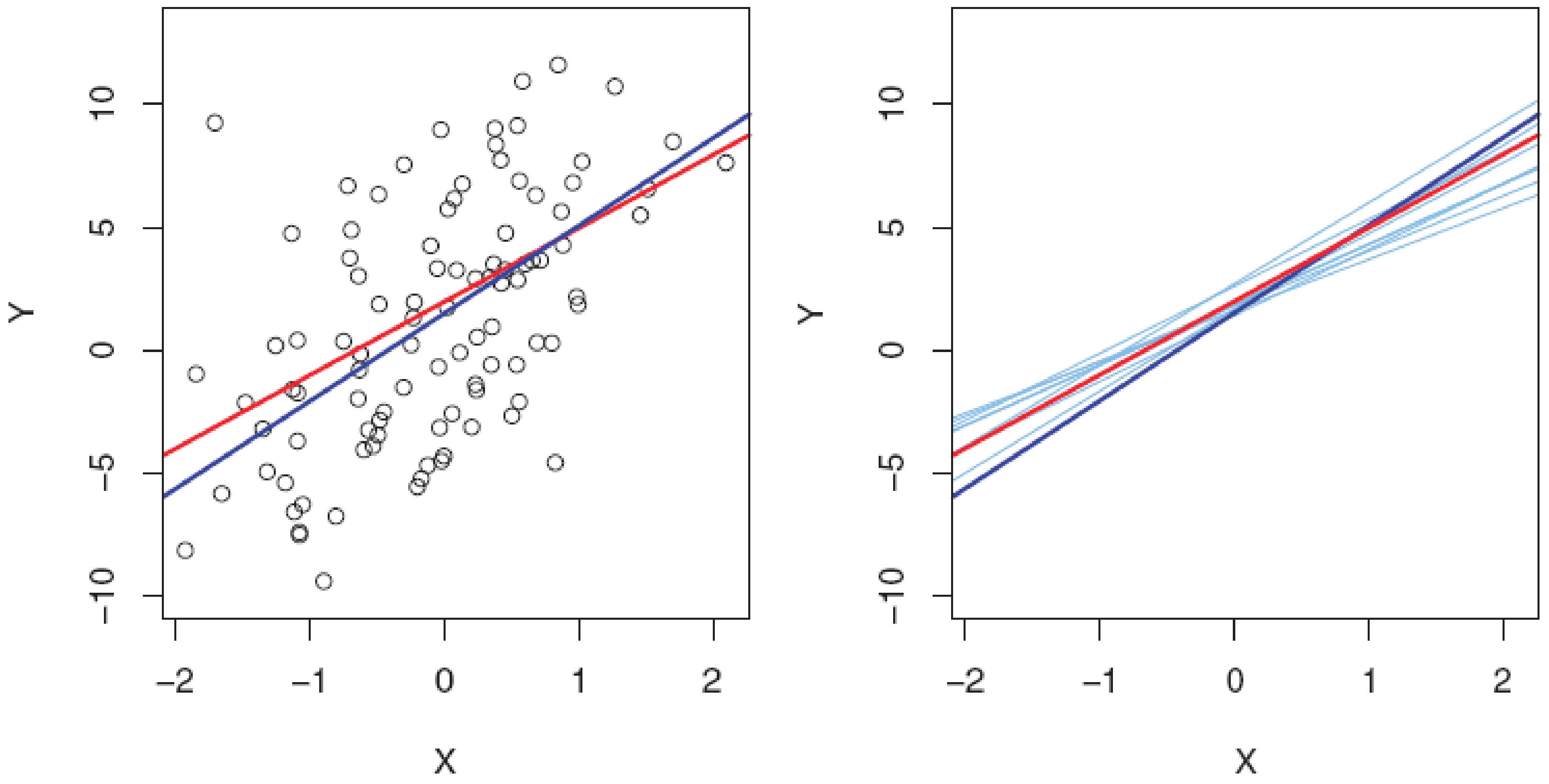
The Greek letter **β** is used to denote the slope ***of the population***

The letter **b** is used to denote the slope ***of the sample***



Population: β_o and β_1

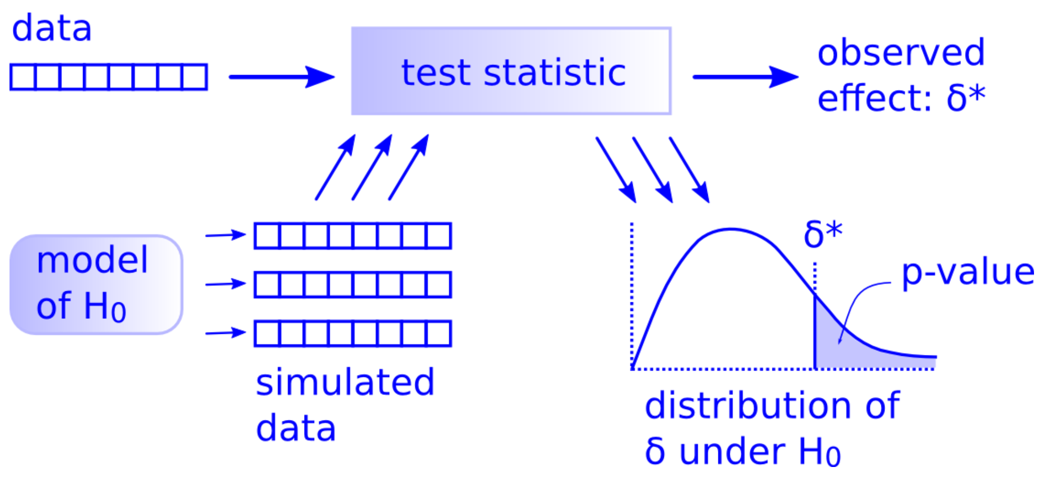
Sample estimates: b_0 and b_1

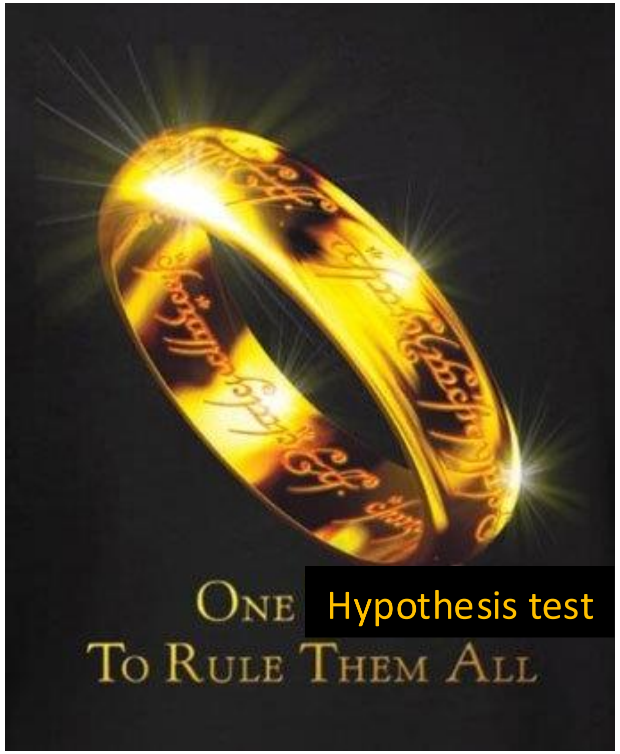


Hypothesis test for regression coefficients

We can run a hypothesis test to see if there is a linear relationship between a feature x, and a response variable y

There is only one hypothesis test!





Hypothesis test for regression coefficients

We can run a hypothesis test to see if there is a linear relationship between a feature x, and a response variable y

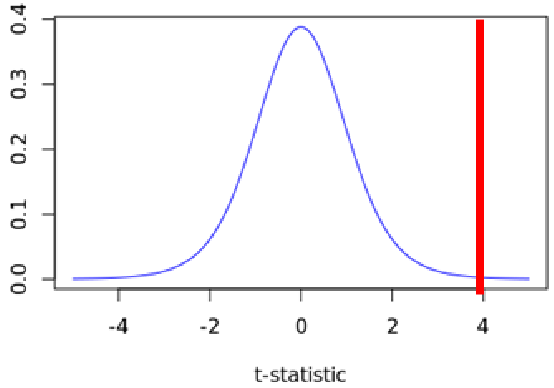
- H_0: $\beta_1 = 0$ (slope is 0, so no linear relationship between x and y
- H_A: $\beta_1 \neq 0$

One type of hypothesis test we can run is based on a t-statistic:

$$t = \frac{b_1 - 0}{\hat{SE}_{b_1}}$$

- The t-statistic comes from a t-distribution with n - k degrees of freedom
  - (If a few conditions are met)

We could also shuffle our data, fit models, and extract b_1 coefficients to create a null distribution

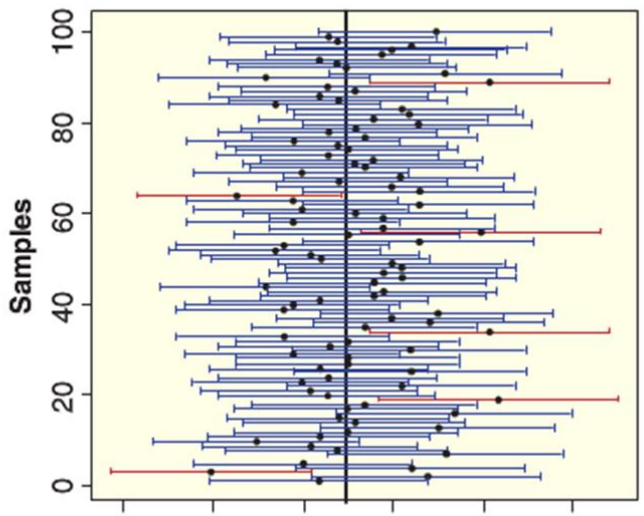


Confidence intervals for regression coefficients

The confidence interval for the slope coefficients: β_1

We can use the statsmodels package to run hypothesis tests and create confidence intervals using “parametric” methods based on t-distributions

```
import statsmodels.api as sm

sm_linear_model = sm.OLS(y_train,
                         X_train_with_constant).fit()

sm_linear_model.summary()
```



Let’s try this in Jupyter!